

Experiment 1: Investigating Hardware Resources and OS Services

Aim: To study how the operating system abstracts hardware resources such as CPU, memory, storage, and I/O devices using Linux commands.

Commands Used

1) Operating System Information

uname -a

Sample Output

Linux Alekhya 6.6.87.2-microsoft-standard-WSL2 x86_64 GNU/Linux

Displays:

Operating System Name

Kernel Version

Architecture

OS Service

The operating system abstracts hardware details and provides a common interface to applications.

2) CPU Information

lscpu

Sample Output: Architecture: x86_64

CPU(s): 8

Model name: Intel Core i5

Displays:

Number of CPUs

CPU architecture

Processor model

OS Service

The operating system abstracts the physical CPU and schedules processes for execution.

3) Storage Information

Lsblk

Sample Output:

NAME	SIZE	TYPE
sda	512G	disk

Displays:

Available disks

Disk partitions

Storage devices

OS Service

The operating system abstracts storage devices as files and directories.

4) Process Information

ps -ef

Sample Output

UID	PID	PPID	CMD
alekhya	321	320	-bash
alekhya	450	321	ps -ef

Displays:

Running processes

Parent Process IDs

Process hierarchy

OS Service

The operating system manages process creation, scheduling, and termination.

5) System Monitoring

top

Observation

Displays:

CPU utilization

Memory utilization

Running processes

System load

OS Service

The operating system continuously allocates CPU time and memory resources.

Report

CPU Abstraction

The operating system hides the complexity of CPU hardware from users and applications. Using scheduling algorithms, it allocates CPU time among multiple processes, creating the illusion that several programs run simultaneously.

Command Used:

lscpu

Memory Abstraction

The operating system manages RAM and allocates memory to processes when required. It also protects one process from accessing another process's memory.

Command Used:

free -h

top

Storage Abstraction

The operating system abstracts storage devices as files and directories. Users do not interact directly with disks; instead, they use the file system.

Command Used:

lsblk

I/O Device Abstraction

The operating system provides device drivers that allow applications to access hardware devices such as keyboards, printers, displays, and disks without knowing hardware details.

Commands Used:

uname -a

lsblk

Experiment-Fork() : Command Execution using fork(), exec(), and wait()

Aim: To develop a C program that demonstrates how a Linux operating system executes a command entered by a user using fork(), exec(), and wait() system calls.

Program 1: One Process (Before fork())

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    printf("PID = %d\n", getpid());
    return 0;
}
```

Only one process exists.

Introduce getpid().

Output : PID = 2456

Program 2: fork() Creates a Child

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    fork();
    printf("Hello\n");
    return 0;
}
```

How many times will "Hello" print?

Output

Hello

Hello

Concept: One process becomes two processes.

Program 3: Parent vs Child

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid == 0)
        printf("I am Child\n");
    else
        printf("I am Parent\n");
    return 0;
}
```

Output

I am Parent

I am Child

Concept

fork() returns 0 in the child.

fork() returns child PID in the parent.

Program 4: Display Both PIDs

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid == 0)
    {
        printf("Child PID = %d\n", getpid());
        printf("Parent PID = %d\n", getppid());
    }
    else
    {
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    return 0;
}
```

Concept

Relationship between parent and child.

Possible Output 2 (Child runs first)

Child PID = 2501

Parent PID = 2500

Parent PID = 2500

Child PID = 2501

Program 5: Parent Waits for Child

```
#include <stdio.h>
```

```
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child Executing\n");
    }
    else
    {
        wait(NULL);
        printf("Parent Executing\n");
    }
    return 0;
}
```

Output

Child Executing

Parent Executing

Concept

Synchronization using wait().

Program 6: Multiple fork() Calls

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    fork();
    fork();
```

```
    printf("PID = %d\n", getpid());  
    return 0;  
}
```

How many processes are created?

Output

Four different PIDs are printed.

Concept

1 fork → 2 processes

2 forks → 4 processes

3 forks → 8 processes

Program 7: Child Executes Another Program (exec())

```
#include <stdio.h>  
#include <unistd.h>  
int main()  
{  
    pid_t pid = fork();  
    if(pid == 0)  
    {  
        printf("Child executing ls...\n");  
        execl("/bin/ls", "ls", "-l", NULL);  
    }  
    else  
    {  
        printf("Parent Waiting...\n");  
    }  
    return 0;  
}
```

Concept

exec() replaces the child process with another program.

Program 10: fork() Inside a Loop (Fun)

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    int i;
    for(i=0;i<3;i++)
        fork();
    printf("PID = %d\n",getpid());
    return 0;
}
```

How many processes are created?

Answer: $2^3 = 8$

Another program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    char cmd[50];
```

```
printf("Enter Linux Command: ");
scanf("%s", cmd);

pid_t pid = fork();

if(pid < 0)
{
    printf("Fork Failed\n");
    return 1;
}
else if(pid == 0)
{
    printf("\nChild Process\n");
    printf("Child PID : %d\n", getpid());
    printf("Parent PID: %d\n", getppid());

    execlp(cmd, cmd, NULL);

    perror("Command Execution Failed");
    exit(1);
}
else
{
    printf("\nParent Process\n");
    printf("Parent PID : %d\n", getpid());
    printf("Child PID : %d\n", pid);

    wait(NULL);

    printf("\nChild Process Completed\n");
}
```

```
    return 0;  
}
```

ls

Output:

Enter Linux Command: ls

Parent Process

Parent PID : 1876

Child PID : 1877

Child Process

Child PID : 1877

Parent PID: 1876

Child Process Completed

```
ritwik@DESKTOP-PK3LAMN:~$ gcc examplefork.c -o examplefork  
ritwik@DESKTOP-PK3LAMN:~$ ./examplefork  
Enter Linux Command: ls  
  
Parent Process  
Parent PID : 5540  
Child PID : 5541  
  
Child Process  
Child PID : 5541  
Parent PID: 5540  
OS-KLU    OSProgram    hello      process  
OSLab     examplefork  hello.c    process.c  
OSSP      examplefork.c helloworld  
  
Child Process Completed  
ritwik@DESKTOP-PK3LAMN:~$ █
```

Explanation:

1) fork()- Creates a new child process.

```
pid_t pid = fork();
```

2) exec()- Replaces the child process with the command entered by the user.

```
execlp(cmd, cmd, NULL);
```

3) wait()- Makes the parent process wait until the child process finishes.

```
wait(NULL);
```

4) getpid()- Returns the Process ID of the current process.

5) getppid()- Returns the Parent Process ID.